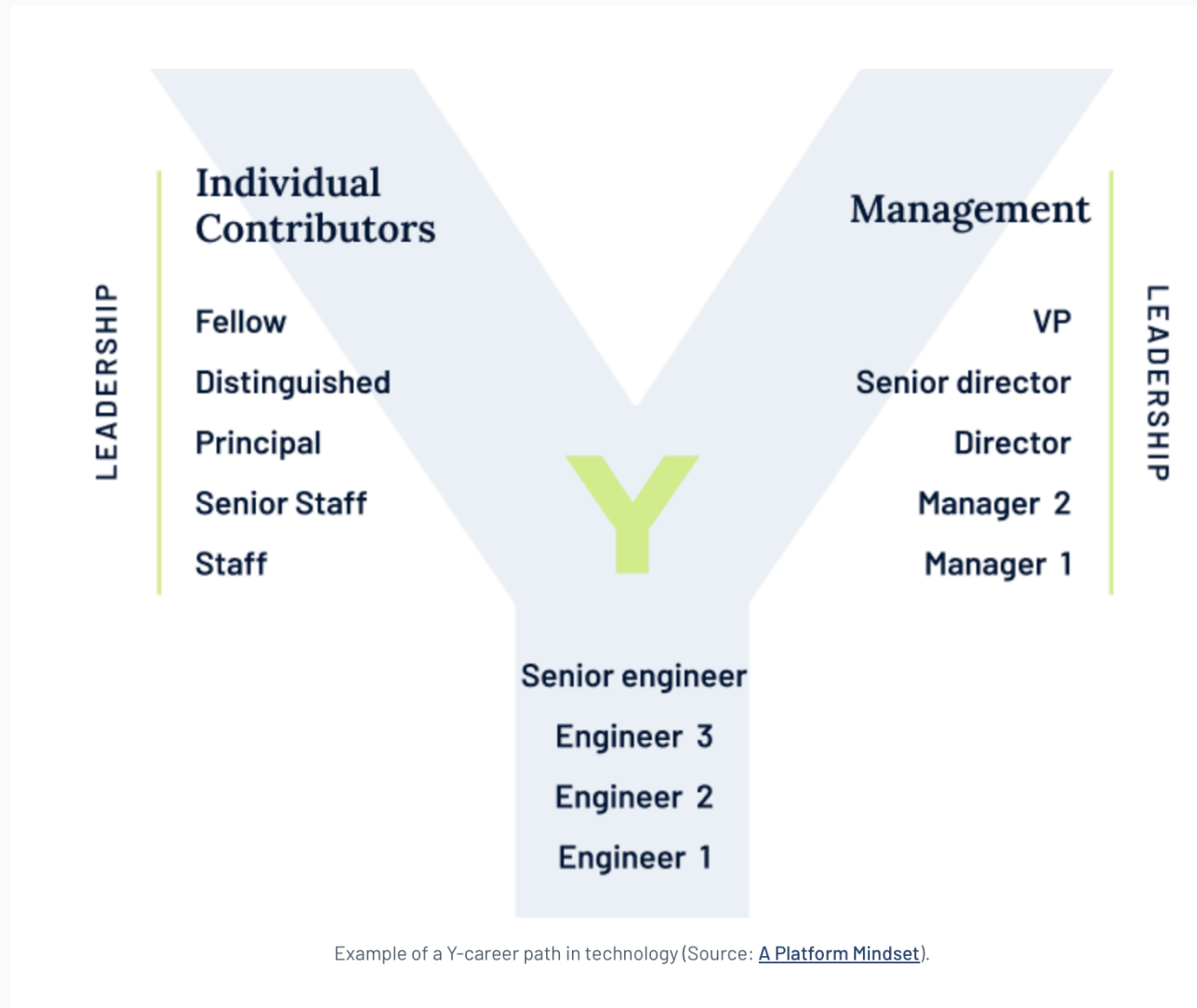


Ciclo de vida de un programa

Clase 3 · 17 de septiembre de 2026 · Aula

Unidad 2 · Fundamentos de programación

Vistazo al mundo real



Objetivo de la sesión

Identificar y aplicar las ocho etapas del ciclo de vida de un programa.

Repasar

Definición y tipos de algoritmo vistos en la clase anterior.

Comprender

Qué ocurre desde que aparece un problema hasta que el software se mantiene.

Aplicar

Recorrer las ocho etapas usando una app de lista de tareas.

Practicar

Analizar en equipos un sistema de control de asistencia.

Del algoritmo al programa

Resolver un problema con una computadora requiere un proceso ordenado, no improvisación.

Algoritmo

La lógica o secuencia de pasos que resuelve un problema.

Ejemplo: pasos para calcular un promedio.

Ciclo de vida del programa

El proceso completo: entender, diseñar, codificar, probar, documentar, implantar y mantener.

Pregunta

¿Qué problemas aparecen si abrimos Python y comenzamos a escribir código sin saber exactamente qué queremos resolver?

Las ocho etapas del ciclo de vida

EXAM

El documento del curso las presenta como un proceso ordenado.

1. Definición del problema

¿Qué debemos resolver?

2. Análisis

Entradas, salidas y restricciones.

3. Diseño de solución

¿Cómo lo resolveremos?

4. Codificación

Traducir el diseño a código.

5. Pruebas y depuración

Comprobar y corregir.

6. Documentación

Explicar el programa.

7. Liberación, despliegue, publicación, (deploy)

Ponerlo en uso real.

8. Mantenimiento

Corregir y mejorar después.

En la práctica

Podemos regresar a etapas anteriores cuando descubrimos errores o cambian las necesidades.

1. Definición del problema

EXAM

Identificar y delimitar QUÉ se va a resolver, sin pensar todavía en cómo.

Definición

Primera etapa: expresar el problema con claridad y límites concretos.

✓ Ejemplo

“Se necesita un programa que calcule el promedio de calificaciones de un estudiante.”

X Contraejemplo

“Hacer un programa útil para la escuela.” Es demasiado vago: no especifica qué hace.

Pregunta

¿Podemos describir claramente la necesidad en una sola oración?

2. Análisis del problema

EXAM

Estudiamos el problema para identificar los datos y las condiciones.

Entrada

¿Qué información necesita?
Ej.: calificaciones.

Salida

¿Qué debe producir?
Ej.: promedio.

Restricciones

¿Qué condiciones deben cumplirse?
Ej.: valores entre 0 y 10.

X No es análisis

Empezar a escribir código sin identificar qué datos recibe el programa ni qué debe producir significa omitir esta etapa.

3. Diseño de la solución

EXAM

Planteamos la lógica independientemente del lenguaje de programación.

Herramientas

Podemos usar:

- pasos numerados
- pseudocódigo
- diagramas
- dibujos de la solución

La pregunta es: ¿cómo resolveremos el problema?

Ejemplo

INICIO

recibir calificaciones

sumar calificaciones

dividir suma entre cantidad

mostrar promedio

FIN

Todavía no necesitamos conocer la sintaxis exacta de Python.

4. Codificación

EXAM

La solución diseñada se traduce a un lenguaje concreto; en este curso, Python.

Diseño

PROMEDIO(calificaciones)

suma $\leftarrow$ sumar calificaciones
promedio $\leftarrow$ suma / cantidad
MOSTRAR promedio

Python

```
calificaciones = [8, 9, 7, 10]
```

```
suma = sum(calificaciones)  
promedio = suma / len(calificaciones)
```

```
print(promedio)
```

Idea

Diseño = lógica · Codificación = implementación concreta.

5. Pruebas y depuración

EXAM

Probamos con distintos datos, incluidos casos límite, y corregimos los errores encontrados.

Caso normal

[8, 9, 7] → ¿da el promedio esperado?

Caso límite

¿Qué sucede con una lista vacía?

Dato inválido

¿Qué sucede con -3 o 15?

Pruebas

Comparamos el resultado esperado con el resultado real.

Depuración

Localizamos la causa del error y modificamos el programa.

6. Documentación

EXAM

Generamos información escrita para explicar qué hace el programa y cómo usarlo o mantenerlo.

Comentarios

Explican decisiones o partes relevantes del código.

Docstrings

Describen funciones, parámetros y resultados.

Manuales

Ayudan a usuarios o a otros desarrolladores.

X Contraejemplo

Código sin comentarios ni explicación, que solamente entiende su autor original: el programa funciona, pero no está adecuadamente documentado.

7. Deploy

EXAM

El programa terminado se instala o publica en el entorno real donde lo utilizarán los usuarios.

App

Publicarla para que los usuarios puedan instalarla.

Sistema escolar

Instalarlo en los equipos o servidor de la institución.

Aplicación web

Subirla a un servidor accesible por los usuarios.

X Contraejemplo

Si el programa solo funciona en la computadora del desarrollador y nunca llega a sus usuarios finales, todavía no ha sido implantado.

8. Mantenimiento

EXAM

Después de la implantación siguen apareciendo necesidades.

Corregir

Errores descubiertos por usuarios.

Actualizar

Cambios en plataformas o dependencias.

Mejorar

Agregar funciones solicitadas.

Adaptar

Responder a nuevas reglas o necesidades.

Ejemplo

Seis meses después de publicar una app, aparece un error en las notificaciones. El equipo lo corrige y publica una actualización.

Ejemplo de principio a fin: app de lista de tareas

EXAM

El mismo caso nos permite ver cómo se conectan las ocho etapas.

1 • Problema

Necesitamos registrar y recordar tareas pendientes.

2 • Análisis

Entradas: tarea/fecha. Salida: lista y estado.

3 • Diseño

Definir agregar, listar y completar tareas.

4 • Codificación

Implementar la lógica y la interfaz.

5 • Pruebas

Probar tareas vacías, muchas tareas y cambios de estado.

6 • Documentación

Explicar uso y estructura del programa.

7 • Implantación

Poner la app a disposición de usuarios.

8 • Mantenimiento

Corregir fallos y agregar mejoras.

Pregunta

Si los usuarios piden recordatorios por correo después de publicar la app, ¿en qué etapa estamos?

¿En qué etapa estamos?

EXAM

Actividad rápida: identifica la etapa antes de revelar la respuesta.

“¿Qué información debe ingresar el usuario?”

ANÁLISIS

“Voy a escribir la función en Python.”

CODIFICACIÓN

“Con una lista vacía el programa falla.”

PRUEBAS / DEPURACIÓN

“Publicamos la nueva versión en el servidor.”

Deploy

“Agregamos un manual para los usuarios”

DOCUMENTACIÓN

“Los usuarios pidieron una nueva función.”

MANTENIMIENTO

Cierre: del problema al software en uso

La codificación es importante, pero es solamente una etapa del proceso.

Antes de programar

Definir → analizar → diseñar

Construir y verificar

Codificar → probar/depurar

Llevarlo al mundo real

Documentar → implantar → mantener

Para la próxima sesión

Tener acceso a una computadora y, si es posible, instalar Python. La siguiente clase será de entorno de Python, conceptos básicos y traductores.